

InsightStream

AI Evaluation (Evals)

Golden Dataset • 15 Hard-Mode Test Cases • Auditor Matrix • Grading Rubric • Regression Protocol

Table of Contents

1. Golden Dataset Schema.....	3
1.1 Master Evaluation Spreadsheet: Column Definitions.....	3
1.2 Error Category Taxonomy.....	4
2. Hard-Mode Test Cases (15 Cases).....	5
2.1 Hallucination Tests (TC-HALL-01 to TC-HALL-03).....	5
2.2 Sarcasm and Irony Tests (TC-SARC-01 to TC-SARC-03).....	6
2.3 Loud User Bias Tests (TC-LOUD-01 to TC-LOUD-02).....	6
2.4 Technical Jargon Tests (TC-JARG-01 to TC-JARG-02).....	7
2.5 Diarization Drift Tests (TC-DIAR-01 to TC-DIAR-02).....	8
2.6 Edge Case Tests (TC-EDGE-01 to TC-EDGE-03).....	8
3. Auditor Performance Matrix.....	9
3.1 Confusion Matrix.....	9
3.2 Metrics Definitions.....	9
3.3 Auditor Failure Investigation Protocol.....	9
4. Grading Rubric: Human-in-the-Loop Quality Scoring.....	10
4.1 Scoring Guidelines for Reviewers.....	10
5. Regression Testing Protocol.....	11
5.1 Evaluation-to-Iteration Loop.....	11
5.2 When to Run the Harness.....	11
5.3 Baseline Management.....	12
5.4 Execution Commands.....	13

1. Golden Dataset Schema

The Golden Dataset is the single source of truth for evaluating InsightStream's extraction quality. Every row represents one test case. Every column captures one measurable dimension. The dataset is version-controlled alongside the codebase.

1.1 Master Evaluation Spreadsheet: Column Definitions

Column	Type	Definition	Example	Populated By
test_id	string	Unique identifier. Format: TC-{category}-{number}	TC-HALL-01	Test author
category	enum	HALLUCINATION, SARCASM, LOUD_USER, JARGON, DIARIZATION, MULTILINGUAL, EMPTY, BOUNDARY, BASELINE	SARCASM	Test author
input_transcript	text	The raw transcript snippet fed to the pipeline	"Speaker 1: Oh great, another login screen..."	Test author
expected_insight_type	enum	FEATURE_REQUEST, PAIN_POINT, NONE, or THEME	NONE	Test author
expected_content	text	What the AI should extract. "NONE" if no insight should be produced.	"No extraction. Sarcasm, not a genuine request."	Test author
expected_confidence_range	string	Acceptable confidence score range	"1-3" or "N/A"	Test author
expected_auditor_result	enum	TRUE (should pass entailment), FALSE (should be rejected), N/A	FALSE	Test author
actual_ai_output	JSON	The raw JSON output from run_agent() for this test case	{feature_requests: [], pain_points: [...]}	Pipeline (automated)
actual_auditor_result	bool	Did the Auditor accept (True) or reject (False) each insight?	False	Pipeline (automated)
pass_fail	enum	PASS if actual matches expected. FAIL if not. Computed automatically.	PASS	Harness (computed)
error_category	enum	On FAIL: FALSE_POSITIVE, FALSE_NEGATIVE, WRONG_TYPE, WRONG_SPEAKER,	FALSE_POSITIVE	Harness or human

		SCORE_DRIFT, PARAPHRASED_QUOTE		
human_quality_score	int 1-5	Manual rating by PM reviewer (see Grading Rubric, Section 4)	4	Human reviewer
notes	text	Free-text field for reviewer observations	"Auditor correctly caught sarcasm"	Human reviewer
run_timestamp	ISO 8601	When this test was last executed	2025-06-15T14:30:00Z	Harness (automated)
model_version	string	Gemini model used for this run	gemini-2.5-flash	Harness (automated)
prompt_hash	string	SHA-256 hash of the Extractor system prompt at time of run	a3f8c2...d91e	Harness (automated)

1.2 Error Category Taxonomy

Error Category	Definition	Severity
FALSE_POSITIVE	AI produced an insight that should not exist. The Auditor should have caught it but did not.	Critical (P0)
FALSE_NEGATIVE	AI missed a real insight, or the Auditor rejected a valid one.	High (P1)
WRONG_TYPE	Insight exists but is miscategorized (e.g., pain point labeled as feature request).	Medium (P2)
WRONG_SPEAKER	Insight attributed to wrong source_id (e.g., Speaker 0 instead of Speaker 1).	High (P1)
SCORE_DRIFT	Confidence score outside the expected range by 3+ points.	Low (P3)
PARAPHRASED_QUOTE	supporting_quote is not verbatim. The LLM rewrote it instead of extracting exactly.	High (P1)

2. Hard-Mode Test Cases (15 Cases)

Each test case targets a specific failure mode. The cases are ordered by category. Every case includes the transcript input, the expected AI behavior, the success criteria, and what a failure looks like.

2.1 Hallucination Tests (TC-HALL-01 to TC-HALL-03)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-HALL-01	Speaker 1: "Our pricing page is confusing. Customers don't understand the tier differences. We lose about 15% at the pricing step."	Pain point about confusing pricing. NO mention of competitors. Auditor rejects any insight about "competitor comparison" or "competitive analysis."	AI fabricates: "User wants a competitor pricing comparison table." This is not in the transcript. Classic hallucination.
TC-HALL-02	Speaker 1: "The API is solid. We use it daily for our batch jobs. No complaints on that front."	No pain point. No feature request. If anything is extracted, confidence should be 1–2 at most. Auditor should reject any negative insight.	AI invents a problem: "User reports API reliability issues." The transcript explicitly says no complaints. Inversion hallucination.
TC-HALL-03	Speaker 1: "We switched from Slack to Teams last quarter." Speaker 0: "How's that going?" Speaker 1: "Fine. Not relevant to this tool though."	No insight. The user explicitly marks this as off-topic. Extractor should skip it. Auditor should reject any Slack/Teams insight.	AI extracts: "User wants Slack-to-Teams migration support." Off-topic content treated as a product need.

2.2 Sarcasm and Irony Tests (TC-SARC-01 to TC-SARC-03)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-SARC-01	Speaker 1: "Oh great, another login screen, just what my life was missing. I love entering my password for the fifth time today."	Pain point: excessive login prompts / poor session persistence. Confidence 7–9. NOT a positive feature request about login screens.	AI extracts as positive: "User appreciates the login experience." Sarcasm read as genuine praise.
TC-SARC-02	Speaker 1: "The loading speed is incredible. I can make coffee, walk the dog, and it's still loading when I get back."	Pain point: extremely slow loading. Confidence 8–10. The sarcastic framing describes a genuine, severe problem.	AI reports: "User praises loading speed" or assigns low confidence (1–4) to a clearly critical issue.
TC-SARC-03	Speaker 1: "The color scheme is nice? I'm kidding. Sort of. Anyway, the real issue is that I can't export my data."	Feature request: data export. Confidence 8–10. NO insight about color scheme. The sarcasm is a throwaway; the export request is the real signal.	AI extracts: "User wants color scheme customization" as a genuine feature request alongside the export request.

2.3 Loud User Bias Tests (TC-LOUD-01 to TC-LOUD-02)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-LOUD-01	Speaker 1 (800 words): Repeats "dashboard is slow" 7 times with different phrasing. Speaker 2 (one sentence): "The billing portal doesn't show itemized invoices. I email finance every month for a breakdown."	Both pain points extracted. In theme synthesis, billing should NOT be ranked below dashboard just because Speaker 1 talked more. Unique participant count = 1 each.	Core Themes ranks "Dashboard Performance" at 9/10 and "Billing" at 3/10 purely because of mention frequency.
TC-LOUD-02	Speakers 1, 2, and 3 each mention "search is broken" once. Speaker 4 mentions "mobile app crashes" 12 times. No other speaker mentions mobile.	Theme synthesis: "Search" should rank higher than "Mobile" because 3 unique participants raised it vs. 1 for mobile. source_id weighting should override frequency.	"Mobile App Stability" ranked as top theme because of raw frequency (12 mentions vs. 3).

2.4 Technical Jargon Tests (TC-JARG-01 to TC-JARG-02)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-JARG-01	Speaker 1: "The SSO via OIDC is flaky. Half the time the SAML assertion fails and we get a 403. Our IT team has to reset the IdP config every week."	Pain point: Single Sign-On / authentication reliability. The Extractor should understand that SSO, OIDC, SAML, 403, and IdP all relate to login/authentication. Confidence 8–10.	AI fails to connect the jargon to a product category. Extracts: "User reports OIDC issue" without mapping to "Login / Authentication." Or skips entirely due to unrecognized acronyms.
TC-JARG-02	Speaker 1: "Our CI/CD pipeline chokes on the webhook payload. The JSON schema validation throws a 422 because you send nested arrays where we expect flat objects."	Pain point: API/webhook response format incompatibility. Feature request: flat JSON schema option. Confidence 9–10 (explicit, technical, actionable).	AI extracts a vague insight: "User has integration issues" with confidence 4–5. The technical specificity is lost.

2.5 Diarization Drift Tests (TC-DIAR-01 to TC-DIAR-02)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-DIAR-01	Speaker 0: "The export feature is terrible. I spend two hours every Friday copying data manually." Speaker 1: "Can you tell me more about that?"	Labels are inverted. Speaker 0 is the participant; Speaker 1 is the interviewer. The Extractor prompt says "IGNORE Speaker 0." Expected: no insights extracted (the real feedback is from Speaker 0, which is ignored).	AI extracts Speaker 0's feedback despite the prompt instruction to ignore it. Or: AI extracts Speaker 1's question as a feature request.
TC-DIAR-02	Speaker 1: "I need bulk export." Speaker 1: "Tell me more." Speaker 1: "Like, CSV for filtered views." Speaker 1: "Interesting, what filters?"	All lines labeled as Speaker 1 (diarization failed completely). The Extractor should still extract the genuine feedback (bulk export, CSV, filtered views) and skip the interviewer questions. Confidence should reflect uncertainty about speaker.	AI treats interviewer questions as user needs: "User wants to know about filters." Or: source_id = Speaker 1 for everything, making anti-bias weighting meaningless.

2.6 Edge Case Tests (TC-EDGE-01 to TC-EDGE-03)

ID	Input Transcript	Expected Result	Failure Looks Like
TC-EDGE-01	Speaker 1: "Le problème principal, c'est que l'export ne marche pas. It just doesn't work. Vraiment frustrant."	Pain point: export is broken. The insight should be extracted despite the French. supporting_quote should preserve the multilingual text exactly as spoken. Confidence 8–10.	AI drops the French content entirely. Or: supporting_quote contains only the English fragment, losing the French context.
TC-EDGE-02	[Empty transcript. No text. Just whitespace.]	Zero insights. Zero API calls. No errors. The pipeline should short-circuit cleanly.	Pipeline crashes, throws an unhandled exception, or calls the Gemini API on empty input.
TC-EDGE-03	Speaker 1: "I want to like this product. I really do. The core concept is great. But the inconsistency is killing us. It feels like the team ships new features without stabilizing existing ones."	Pain point: product reliability / stability. This is positive framing wrapped around a serious complaint. Confidence 7–9. NOT a positive feature request.	AI extracts: "User likes the product concept" as a positive signal while ignoring the core complaint about instability.

3. Auditor Performance Matrix

The Auditor is InsightStream's last line of defense. This section defines how we measure whether it does its job.

3.1 Confusion Matrix

	Auditor Says: PASS	Auditor Says: REJECT
Insight IS Grounded (Reality)	True Positive (TP): Correct acceptance. The insight is real and the Auditor let it through.	False Negative (FN): Incorrect rejection. The insight is real but the Auditor blocked it. User can recover via search.
Insight is NOT Grounded (Reality)	False Positive (FP): Hallucination that slipped through. CRITICAL FAILURE. P0 severity.	True Negative (TN): Correct rejection. The Auditor blocked a hallucination. Working as designed.

3.2 Metrics Definitions

Metric	Formula	Target	Alert Threshold
Precision	$TP / (TP + FP)$	100%	Any FP = P0 incident
Recall	$TP / (TP + FN)$	$\geq 75\%$	< 60% for 3 consecutive runs
Accuracy	$(TP + TN) / \text{Total}$	$\geq 85\%$	< 75%
F1 Score	$2 \times (P \times R) / (P + R)$	≥ 0.85	< 0.70
FP Count (absolute)	Count of FP across all test cases	0	≥ 1 = immediate investigation

3.3 Auditor Failure Investigation Protocol

When a False Positive is detected (a hallucination that passed the Auditor):

1. Stop. Do not ship any prompt or model changes until the root cause is identified.
2. Extract the specific (statement, segment) pair that the Auditor incorrectly accepted.
3. Reproduce: run `verify_entailment()` on the pair in isolation. Confirm the failure is consistent, not a one-time API artifact.
4. Classify: is the failure due to (a) the entailment prompt being too permissive, (b) the segment window missing context, (c) a Gemini model behavior change, or (d) a malformed statement construction?
5. Fix: adjust the relevant component (prompt, window size, statement construction).
6. Verify: re-run the full evaluation harness. Confirm FP count = 0 and recall has not degraded.
7. Document: add the failure case to the Golden Dataset as a permanent regression test.

4. Grading Rubric: Human-in-the-Loop Quality Scoring

During manual review of evaluation results, a PM scores each AI-extracted insight on a 1–5 scale. This score is recorded in the `human_quality_score` column of the Golden Dataset.

Score	Label	Criteria	Action
5	Production-Ready	The insight is correct, specific, and actionable. The supporting quote is verbatim and relevant. The confidence score matches the rubric. A PM would export this to Jira without editing a single word.	Auto-approve. No changes needed.
4	Good with Minor Edit	The insight captures the right idea but the wording could be tighter. Or the confidence score is off by 1–2 points. The supporting quote is correct. A PM would approve after a quick rewrite of the <code>user_need</code> or <code>proposed_solution</code> .	Approve after minor cleanup.
3	Partially Correct	The insight identifies a real topic but gets the framing wrong. Example: extracts a feature request when it should be a pain point. Or the confidence score is off by 3+ points. The quote is correct but the analysis is incomplete.	Review carefully. May need reclassification.
2	Misleading	The insight distorts the user's intent. The quote is taken out of context, or the <code>proposed_solution</code> doesn't follow from the <code>user_need</code> . A PM who didn't read the transcript might make a wrong decision based on this.	Reject. Flag for prompt investigation.
1	Hallucination	The insight has no basis in the transcript. The supporting quote is fabricated or irrelevant. The Auditor should have caught this. If it reached the user, this is a system failure.	Reject. P0 incident. Auditor investigation.

4.1 Scoring Guidelines for Reviewers

- Score the insight, not the transcript. A poorly worded transcript that produces a correct insight still scores 5.
- Score based on what the PM would do with it. Would they export it as-is (5)? Edit it (4)? Hesitate (3)? Reject it (2)? Report a bug (1)?
- Do not adjust scores for confidence. The confidence score is a separate evaluation dimension. A score-5 insight with a wrong confidence number still scores 5 for quality; the confidence drift is tracked separately in the `error_category` column.
- When in doubt between two scores, choose the lower one. The rubric should err on the side of strictness.

5. Regression Testing Protocol

The evaluation harness runs as a mandatory gate before any change to the system reaches production. This section defines the exact process.

5.1 Evaluation-to-Iteration Loop

1. CHANGE — Developer modifies prompt, schema, model, or pipeline code

▼ triggers

2. RUN HARNESS — Execute all 15+ test cases against the modified system

▼ produces

3. COMPARE — Diff results against the previous baseline run

▼ if FP > 0 or recall < threshold

4. BLOCK — Change cannot ship. Root cause investigation required.

▼ if all metrics pass

5. SHIP — Update baseline. Tag the run. Deploy.

▼ feeds back into

6. MONITOR — Track production Manual Override Rate for 7 days post-deploy

5.2 When to Run the Harness

Change Type	Harness Run	Minimum Scope	Approval Required
System prompt edit (EXTRACTOR, DEDUP, SYNTHESIZER, or ENTAILMENT)	Required	All 15 hard-mode cases + full baseline	PM + Engineer sign-off
Pydantic schema change	Required	All cases (schema affects output shape)	Engineer sign-off
Model version change (e.g., Flash update)	Required	All cases + human_quality_score review on 5 samples	PM + Engineer + Prod owner
Pipeline code change (node logic, chunking, dedup)	Required	All cases	Engineer sign-off
UI-only change (no pipeline impact)	Not required	N/A	Designer sign-off
New test case added to Golden Dataset	Required	Run the new case + verify existing baseline unchanged	Engineer sign-off
Scheduled weekly regression (no code changes)	Required	Full baseline. Catches silent model behavior changes from Google.	Automated (alert on failure)

5.3 Baseline Management

What is a baseline? The most recent harness run where all metrics passed. Stored as a JSON snapshot: {test_id: pass_fail, metrics: {precision, recall, accuracy}, timestamp, prompt_hash, model_version}.

How baselines are updated: After a successful harness run that includes a code change, the new results become the baseline. The previous baseline is archived (never deleted).

Baseline drift detection: If a weekly regression run produces different results from the baseline without any code changes, this indicates a silent model behavior change from Google. This triggers the same investigation protocol as a P1 (recall degradation) or P0 (precision failure).

5.4 Execution Commands

The harness is executed via pytest. These are the exact commands:

```
# Full harness (all test cases + metrics report):
export GEMINI_API_KEY=your_key
pytest test_insightstream.py test_insightstream_hard.py -v -s

# Auditor-only evaluation (precision/recall):
pytest test_insightstream_hard.py::TestHardAuditorEval -v -s

# Ground-truth coverage check:
pytest test_insightstream_hard.py::TestHardGroundTruthCoverage -v -s

# Unit tests only (no API calls, fast gate):
pytest test_insightstream.py test_insightstream_hard.py -m "not llm" -v
```

CI integration: The unit test suite (69 tests, no API calls) should run on every git push. The full LLM harness runs nightly or on-demand before releases. The weekly regression run is scheduled via cron or CI scheduler.